

6.1.2 Part 2: Implementation - The XML Structure

1. The Fundamentals: Link and Joint

A **Link** is a physical piece with three tags:

- **Visual:** What we see in RViz.
- **Collision:** What the physics engine uses to detect if you hit a wall.
- **Inertial:** The mass and weight distribution (physics).

A **Joint** connects two links. Common types include **fixed** (LiDAR to Chassis), **continuous** (Wheels), and **revolute** (Steering hinges).

2. The Professional Approach: Xacro Macros

File: *urdf/racecar.xacro*

```

1  <?xml version="1.0"?>
2  <robot xmlns:xacro="http://www.ros.org/wiki/xacro" name="
    fayoum_racer">
3
4      <xacro:property name="wheel_radius" value="0.05" />
5      <xacro:property name="wheel_width" value="0.04" />
6      <xacro:property name="chassis_length" value="0.4" />
7
8      <xacro:macro name="wheel" params="prefix x_reflect y_reflect">
9          <link name="${prefix}_wheel">
10             <visual>
11                 <geometry>
12                     <cylinder radius="${wheel_radius}" length="${
13                         wheel_width}" />
14                 </geometry>
15                 <material name="black"><color rgba="0 0 0 1"/></material>
16             </visual>
17             <collision>
18                 <geometry>
19                     <cylinder radius="${wheel_radius}" length="${
20                         wheel_width}" />
21                 </geometry>
22             </collision>
23         </link>
24
25         <joint name="${prefix}_wheel_joint" type="continuous">
26             <parent link="base_link"/>
27             <child link="${prefix}_wheel"/>
28             <origin xyz="${x_reflect * chassis_length/2} ${y_reflect *
29                 0.15} 0" rpy="${pi/2} 0 0"/>
30             <axis xyz="0 0 1"/>
31         </joint>
32     </xacro:macro>

```

```

31 <xacro:wheel prefix="front_left" x_reflect="1" y_reflect="1"
    />
32 <xacro:wheel prefix="front_right" x_reflect="1" y_reflect="-1"
    />
33 <xacro:wheel prefix="rear_left" x_reflect="-1" y_reflect="1"
    />
34 <xacro:wheel prefix="rear_right" x_reflect="-1" y_reflect="-1"
    />
35 </robot>

```

6.1.3 Part 3: Visualizing the Result

To see this car in RViz2, we need `robot_state_publisher`. It takes the Xacro, parses it, and broadcasts the static TF transforms (`base_link` → `wheel_link`).

Implementation: The Launch File

File: `launch/rsp.launch.py`

```

1  import os
2  from ament_index_python.packages import
    get_package_share_directory
3  from launch import LaunchDescription
4  from launch_ros.actions import Node
5  import xacro
6
7  def generate_launch_description():
8      # 1. Path to the xacro file
9      pkg_path = get_package_share_directory('my_robot_description'
10      )
11      xacro_file = os.path.join(pkg_path, 'urdf', 'racecar.xacro')
12
13      # 2. Process Xacro to generate raw XML
14      robot_description_config = xacro.process_file(xacro_file).
15      toxml()
16
17      # 3. Create the Robot State Publisher node
18      node_robot_state_publisher = Node(
19          package='robot_state_publisher',
20          executable='robot_state_publisher',
21          output='screen',
22          parameters=[{'robot_description':
23                      robot_description_config}]
24      )
25
26      return LaunchDescription([node_robot_state_publisher])

```

6.1.4 Part 4: Best Practices & Pitfalls (CRITICAL)

Critical URDF/Xacro Pitfalls

- **The "Inertial" Pitfall:** If you set a mass of 0 for any link, Gazebo will either crash or your robot will fly into the air. Every moving part must have a non-zero inertial tag.
- **The Orientation Trap:** In URDF, cylinders are created along the Z-axis. To make them look like wheels, you must rotate the visual and collision tags by $\pi/2$ (90 degrees).
- **Mesh Management:** Always use `.dae` (Collada) for visual meshes and keep file paths relative using the `package://` syntax.

6.2 Module 6.2: `ros2_control` (The Muscle System)

6.2.1 Part 1: The "Why" & The "How" (The Ecosystem)

The "Why"

If you write a custom script to talk to a specific motor driver over Serial/USB, your code is "locked" to that hardware. If you switch to a different motor driver brand, your navigation stack breaks.

`ros2_control` provides a **Hardware Abstraction Layer (HAL)**. It separates the "what" (e.g., "drive at 2.0 m/s") from the "how" (e.g., "send these specific bits over a CAN bus").

The "How" (Architecture Level)

- **Controller Manager (The Brain):** A node that loads and switches between different controllers (like an Ackermann Steer controller) at runtime.
- **Resource Manager (The Orchestrator):** The internal logic that links the Controllers to the physical Hardware.
- **Hardware Interface (The Muscle):** A C++ plugin that talks to your specific hardware (or the Gazebo simulator). It reads "Command" interfaces (outputs) and writes to "State" interfaces (inputs like encoders).

Real-world Autonomous Racing Use Cases

- **Ackermann Steering Controller:** For our team, we use a standard Ackermann controller. It handles the geometry of turning the front wheels while driving the rear ones, converting a single "Linear Velocity" and "Steering Angle" into four individual wheel speeds.
- **Seamless SITL/HITL:** You use a `mock_hardware` interface for Software-In-The-Loop (SITL) testing and swap to a `racing_car_hardware` interface for the physical race.

6.2.2 Part 2: The URDF Integration

To enable `ros2_control`, we must tell the URDF which joints are "actuated" (powered by motors).

File: `urdf/ros2_control.xacro`

```

1 <ros2_control name="RealRobot" type="system">
2   <hardware>
3     <plugin>gazebo_ros2_control/GazeboSystem</plugin>
4   </hardware>
5
6   <joint name="front_left_steer_joint">
7     <command_interface name="position">
8       <param name="min">-0.5</param>
9       <param name="max">0.5</param>
10    </command_interface>
11    <state_interface name="position"/>
12  </joint>
13
14  <joint name="rear_left_wheel_joint">
15    <command_interface name="velocity">
16      <param name="min">-10</param>
17      <param name="max">10</param>
18    </command_interface>
19    <state_interface name="velocity"/>
20    <state_interface name="position"/>
21  </joint>
22 </ros2_control>
23
24 <gazebo>
25   <plugin name="gazebo_ros2_control" filename="
26     libgazebo_ros2_control.so">
27     <parameters>$(find my_robot_controller)/config/controllers.
28       yaml</parameters>
29   </plugin>
30 </gazebo>

```

6.2.3 Part 3: The YAML Configuration

Now we define the high-level controllers that will "ride" on top of that hardware.

File: `config/controllers.yaml`

```

1 controller_manager:
2   ros__parameters:
3     update_rate: 100  # Hz - How fast the control loop runs
4
5     joint_state_broadcaster:
6       type: joint_state_broadcaster/JointStateBroadcaster
7
8     ackermann_steering_controller:

```

```

9         type: ackermann_steering_controller/
            AckermannSteeringController
10
11 ackermann_steering_controller:
12   ros__parameters:
13     rear_wheel_names: ['rear_left_wheel_joint', '
                        rear_right_wheel_joint']
14     front_steer_names: ['front_left_steer_joint', '
                        front_right_steer_joint']
15
16     wheelbase: 0.3
17     track_width: 0.2
18     wheel_radius: 0.05
19
20     publish_rate: 50.0
21     base_frame_id: base_link

```

6.2.4 Part 4: Best Practices & Pitfalls (CRITICAL)

Critical Control Loop Pitfalls

- **The Real-Time Loop Danger:** The Hardware Interface's `read()` and `write()` functions are executed within a **Real-Time Thread**.
Pitfall: Adding `printf()` or `RCLCPP_INFO()` inside these functions. These are "blocking" operations. If a log takes 5ms, you have broken a 100Hz loop (10ms cycle).
Best Practice: Only do mathematical bit-shifting or memory-mapped I/O in the hardware interface.
- **The "Ghost State" Pitfall:** If your hardware interface doesn't update the `state_interface` (encoders), controllers think the car isn't moving. They will ramp power to 100% to compensate, and the car will launch uncontrollably once it gains traction.
- **Standardized Interfaces:** Always use standard SI units: **Radians** for position and **Radians/Second** for velocity. Never use RPM or Degrees.

6.3 Module 6.3: Simulation Context (Gazebo Integration)

6.3.1 Part 1: The "Why" & The "How" (The Physics Bridge)

The "Why"

Simulation is your "Digital Proving Ground." It allows for **Software-In-The-Loop (SITL)** testing, where the exact same binaries that will run on the car's onboard computer are tested against a simulated physics engine.

- **Physics Validation:** Does the car flip when turning at 5 m/s?
- **Sensor Validation:** Does the LiDAR see the traffic cones?

- **Safety Logic:** Does the emergency brake trigger when a virtual obstacle appears?

The "How" (Architecture Level)

Gazebo is a standalone physics engine. To make it "ROS-aware," we use the `gazebo_ros2_control` plugin.

1. **Gazebo** calculates the gravity, friction, and collisions.
2. **gazebo_ros2_control** acts as a "Hardware Abstraction." To your ROS 2 controllers, Gazebo looks exactly like a real motor driver.
3. The plugin reads the joint commands from ROS 2, applies virtual forces to the Gazebo links, reads the resulting "simulated encoder" positions, and pushes them back to ROS 2.

6.3.2 Part 2: The Simulation URDF Tweaks (Adding Sensors)

To make the car "see" in Gazebo, we must attach sensor plugins to our URDF links.

File: `urdf/gazebo_sensors.xacro`

```

1  <gazebo reference="lidar_link">
2    <sensor name="lidar" type="ray">
3      <pose>0 0 0 0 0 0</pose>
4      <visualize>true</visualize>
5      <update_rate>10</update_rate>
6      <ray>
7        <scan>
8          <horizontal>
9            <samples>360</samples>
10           <min_angle>-3.14</min_angle>
11           <max_angle>3.14</max_angle>
12         </horizontal>
13       </scan>
14       <range>
15         <min>0.3</min>
16         <max>12.0</max>
17       </range>
18     </ray>
19     <plugin name="gazebo_ros_laser" filename="
20       libgazebo_ros_ray_sensor.so">
21       <ros>
22         <remapping>~/out:=scan</remapping>
23       </ros>
24       <output_type>sensor_msgs/LaserScan</output_type>
25       <frame_name>lidar_link</frame_name>
26     </plugin>
27   </sensor>
28 </gazebo>

```

6.3.3 Part 3: The Spawn Launch File (Bringing it to Life)

This launch file orchestrates the boot sequence: starting Gazebo, spawning the entity, and loading the controllers in a deterministic order.

File: *launch/sim.launch.py*

```

1  import os
2  from ament_index_python.packages import
    get_package_share_directory
3  from launch import LaunchDescription
4  from launch.actions import IncludeLaunchDescription,
    ExecuteProcess, RegisterEventHandler
5  from launch.event_handlers import OnProcessExit
6  from launch.launch_description_sources import
    PythonLaunchDescriptionSource
7  from launch_ros.actions import Node
8
9  def generate_launch_description():
10     # 1. Include the Robot State Publisher
11     rsp = IncludeLaunchDescription(
12         PythonLaunchDescriptionSource([os.path.join(
13             get_package_share_directory('my_robot_description'),
14             'launch', 'rsp.launch.py'
15         )]), launch_arguments={'use_sim_time': 'true'}.items()
16     )
17
18     # 2. Start Gazebo
19     gazebo = IncludeLaunchDescription(
20         PythonLaunchDescriptionSource([os.path.join(
21             get_package_share_directory('gazebo_ros'), 'launch',
22             'gazebo.launch.py'
23         )]),
24     )
25
26     # 3. Spawn the robot
27     spawn_entity = Node(package='gazebo_ros', executable='
28         spawn_entity.py',
29         arguments=['-topic', 'robot_description',
30                 '-entity', 'fayoum_racer',
31                 '-z', '0.1'],
32         output='screen')
33
34     # 4. Load Controllers (Delayed via Event Handlers)
35     load_jsb = ExecuteProcess(
36         cmd=['ros2', 'control', 'load_controller', '--set-state',
37             'active', 'joint_state_broadcaster']
38     )
39
40     load_ackermann = ExecuteProcess(
41         cmd=['ros2', 'control', 'load_controller', '--set-state',
42             'active', 'ackermann_steering_controller']
43     )

```

```
39
40     return LaunchDescription([
41         rsp, gazebo, spawn_entity,
42         RegisterEventHandler(OnProcessExit(target_action=
43             spawn_entity, on_exit=[load_jsb])),
44         RegisterEventHandler(OnProcessExit(target_action=load_jsb
45             , on_exit=[load_ackermann])),
46     ])
```

6.3.4 Part 4: Best Practices & Pitfalls (CRITICAL)

Critical Simulation Pitfalls

- **The "Real-Time Factor" (RTF) Nightmare:** If Gazebo is lagging (RTF < 1.0), your perception algorithms will perceive the car as moving faster than it is.
The Fix: Always run nodes with `use_sim_time:=true`.
- **The "Exploding Robot":** If wheels spawn inside the ground, physics restorative forces will launch the car into space.
Best Practice: Always spawn the robot a few centimeters above the ground (e.g., `-z 0.1`).
- **Visualizer Performance:** `<visualize>true</visualize>` is great for debugging but heavy on the GPU. Disable it for long-duration testing to maintain a high RTF.

CHAPTER 7

Phase 7: The Autonomous Stack

7.1 Module 7.1: Sensor Fusion & State Estimation (EKF)

7.1.1 Part 1: The "Why" & The "How" (The Kalman Filter)

The "Why"

Autonomous racing is a game of millimeters. Raw data from individual sensors is often insufficient:

- **Wheel Odometry** is excellent for short distances but suffers from integration drift. If a wheel slips by just 1%, that error compounds indefinitely.
- **IMU (Inertial Measurement Unit)** provides high-frequency updates on rotation and acceleration but is inherently noisy and prone to bias.

Sensor Fusion combines these imperfect sources to produce an estimate more accurate than any single sensor could provide alone.

The "How" (The Kalman Filter)

The Extended Kalman Filter (EKF) operates in a recursive two-step loop:

1. **Predict:** Using the vehicle's last known velocity and a motion model, the EKF "guesses" the next state.
2. **Update:** The EKF compares this guess against actual sensor readings (Odometry/IMU) and calculates a **Kalman Gain**—a weighting factor that decides how much to trust the prediction versus the sensors based on their defined **Covariance** (uncertainty).

Real-world Autonomous Racing Use Cases

- **Slip Compensation:** If rear wheels lose traction during a high-G turn, the EKF sees the velocity spike but notices the IMU accelerometer does not show corresponding force. it "distrusts" the encoders to maintain a realistic pose.